Probeklausur (überarbeitet)

Bearbeitungszeit 90 Minuten

Name	**Probeklausur**
Vorname	**.**
Matrikelnummer	**.**
Datum	**22.01.2008**

Bei der Klausur sind **keine Hilfsmittel** (Skripten, Taschenrechner, etc.) erlaubt. Papier wird gestellt. Benutzen Sie nur die **ausgeteilten**, zusammengehefteten Blätter für Ihre Antworten. Schreiben Sie mit blauem oder schwarzem Kugelschreiber, Füller, Bleistift, etc. ist nicht erlaubt.

1 Abstrakte Beschreibung	14	
2 Logik	10	
3 Zahlen	14	
4 Algorithmen	22	
5 Wachstum von Funktionen	16	
6 Abstrakte Datentype	10	
7 Java	04	
Summe	**90**	

Note	

1 Abstrakte Beschreibung (1/2)

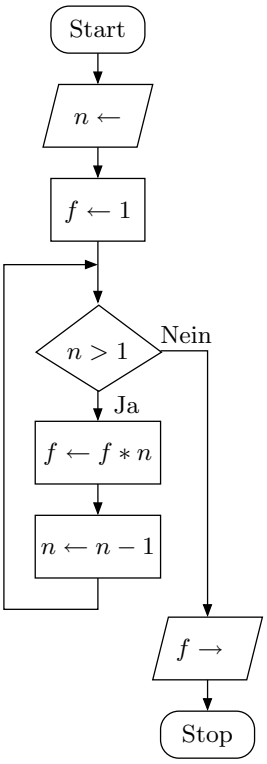
1. Gegeben ist der folgenden Java-Quelltext der Methoden `fac(n)` zur Berechnung der Fakultät von n.

```
1  public int fac(int n) {
2    int f = 1;
3    while (n > 1) {
4      f = f * n;
5      n = n - 1;
6    }
7    return f;
8  }
```

   Zeichnen Sie das Flussdiagramm der Methoden `fac`. (4 Punkte)

Lösung.

1. Flussdiagramm



1 Abstrakte Beschreibung (2/2)

2. Geben Sie über dem Alphabet der Ziffern von 0 bis 9 eine EBNF (erweiterte Backus-Naur-Form) Beschreibung für die Sprache aller natürlichen Zahlen an, die durch 4 teilbar sind. (4 Punkte)

3. Es sind reguläre Audrücke definiert für das leere Wort (ε), die Konkatenation (Hintereinanderstellen), die Alternative ($|$) und beliebig viele Vorkommen ($*$).

   Für reguläre Ausdrücke gilt Folgendes.

   - $*$ bindet stärker als die Konkatenation, Beispiel. $rs* = r(s*)$.
   - Die Konkatenation bindet stärker als $|$. Beispiel. $(rs|ab) = ((rs)|(ab))$.

   Der reguläre Ausdruck $r?$ soll kein oder ein Vorkommen, der reguläre Ausdruck $r+$ soll beliebig viele, aber mindestens ein, Vorkommen von r beschreiben.

   Geben Sie Definitonen von $r?$ und $r+$ mit den bereits definierten regulären Ausdrücken an. (6 Punkte)

Lösung.

2.
```
<S>  ::= "4" | "8"
         | <V0> "0" | <V0> "4" | <V0> "8" | <V1> "2" | <V1> "6"
<V0> ::= "2" | "4" | "6" | "8"
         | <T> "0" | <T> "2" | <T> "4" | <T> "6" | <T> "8"
<V1> ::= "1" | "3" | "5" | "7" | "9"
         | <T> "1" | <T> "3" | <T> "5" | <T> "7" | <T> "9"
<T> ::=  ("1" | ... | "9") { "0" | "1" | ... | "9"}
```

3. $r? = (\varepsilon \mid r)$

   $r+ = (r\ r*)$.

2 Logik

Folgende Gesetze (Rechenregeln) für Operatoren auf Wahrheitswerten können als bekannt vorausgesetzt werden.

Neutralitäts-	Extremal-	Komplementär-	Kommutativ-	Idempotenz-
$x \vee 0 = x$	$x \vee 1 = 1$	$x \vee \neg x = 1$	$x \vee y = y \vee x$	$x \vee x = x$
$x \wedge 1 = x$	$x \wedge 0 = 0$	$x \wedge \neg x = 0$	$x \wedge y = y \wedge x$	$x \wedge x = x$
				$\neg\neg x = x$

Assoziativ-	Distributiv-
$x \vee (y \vee z) = (x \vee y) \vee z$	$x \vee (y \wedge z) = (x \vee y) \wedge (x \vee z)$
$x \wedge (y \wedge z) = (x \wedge y) \wedge z$	$x \wedge (y \vee z) = (x \wedge y) \vee (x \wedge z)$

Hinweis. Der $\neg$ Operator bindet stärker als die Operatoren $\vee$, $\wedge$, d.h. $\neg\ x \vee y = (\neg x) \vee y$ und $\neg\ x \wedge y = (\neg x) \wedge y$. Die Operatoren $\vee$, $\wedge$ binden gleichstark, deshalb sind hier Klammern, die die richtige Auswertungsreihenfolge bestimmen, zwingend erforderlich (Beispiel, Distributivgesetz).

1. Zeigen Sie, mit Hilfe einer ausführlichen Wahrheitswertetafel, dass folgendes De-Morgansche-Gesetz gilt. (4 Punkte)

$$\neg(x \vee y) = \neg x \wedge \neg y$$

2. Betrachten Sie die Absorptionsgesetze.

$$x \vee (x \wedge y) = x$$
$$x \wedge (x \vee y) = x$$

   a) Zeigen Sie, durch Anwendung der passenden bekannten Gesetze, dass aus der Gültigkeit eines Absorptionsgesetze das andere folgt. Geben Sie bei jedem Schritt das angewendete Gesetz an. (3 Punkte)

   b) Zeigen Sie die Gültigkeit eines Absorptionsgesetzes durch Anwendung der passenden bekannten Gesetze. Geben Sie bei jedem Schritt das angewendete Gesetz an. (3 Punkte)

      Hinweis. Führen Sie eine Fallunterscheidung mit $y = x$ und $y = \neg x$ durch. |

Lösung.

1. De-Morgansches-Gesetz

x	y	$x \vee y$	$\neg(x \vee y)$	$\neg x$	$\neg y$	$\neg x \wedge \neg y$
0	0	0	**1**	1	1	**1**
0	1	1	**0**	1	0	**0**
1	0	1	**0**	0	1	**0**
1	1	1	**0**	0	0	**0**

2. a) Gilt ein Absorptionsgesetz, dann gilt auch das andere.

$$\begin{aligned} x \vee (x \wedge y) &= (x \vee x) \wedge (x \vee y) && \text{Distributiv-} \\ &= x \wedge (x \vee y) && \text{Idempotenz-} \end{aligned}$$

b) Fall $x = y$.

$$\begin{aligned} x \vee (x \wedge y) &= x \vee (x \wedge x) && y = x \\ &= x \vee x && \text{Idempotenz-} \\ &= x && \text{Idempotenz-} \end{aligned}$$

Fall $x = \neg y$.

$$\begin{aligned} x \vee (x \wedge y) &= x \vee (x \wedge \neg x) && y = \neg x \\ &= x \vee 0 && \text{Komplementär-} \\ &= x && \text{Neutralitäts-} \end{aligned}$$

Bemerkung. Es gibt einen *Exkurs, Äquivalenzumformung* als Anhang der Probeklausur.

3 Zahlen

1. Rechnen Sie von einem Zahlensytem in das andere um. Der Rechenweg ist Teil der Lösung.

   a) $(377)_8 = (?)_{10}$ (2 Punkte)

   b) $(3200)_{10} = (?)_{16}$ (2 Punkte)

   c) $(1001011001)_2 = (?)_8$ (2 Punkte)

   d) $(E2A0)_{16} = (?)_2$ (2 Punkte)

2. Betrachten Sie vorzeichenbehaftete Dualzahlen der Länge n Bit, wobei für die Darstellung der negativen Zahlen das Basiskomplement (= Zweierkomplement) verwendet wird.

   a) Erläutern Sie, wie zwei solche vorzeichenbehafteten Dualzahlen addiert werden. (4 Punkt)

   b) Rechnen Sie exemplarisch $\texttt{01110}_2 - \texttt{00111}_2$ mit $n = 5$. (2 Punkt)

Lösung.

1. a) Ausmultiplizieren.

$$377_8 = 3_{10} \cdot 8_{10}^2 + 7_{10} \cdot 8_{10} + 7_{10} = 255_{10}$$

b) Dividierende Umrechung.

$$\begin{aligned} 3200_{10}/16 = 200_{10} \quad & \text{Rest} \quad a_0 = 0 \\ 200/16 = 12 \quad & \text{Rest} \quad a_1 = 8 \\ 12/16 = 0 \quad & \text{Rest} \quad a_2 = 12_{10} = \mathrm{C}_{16} \end{aligned}$$

$$3200_{10} = \mathrm{C80}_{16}$$

c) Zerlegen

$$\begin{aligned} (1001011001)_2 &= (100)_2^3 \cdot (001)_2 + (100)_2^2 \cdot (001)_2 + (100)_2 \cdot (011)_2 + (001)_2 \\ &= (10)_8^3 \cdot (1)_8 + (10)_8^2 \cdot (1)_8 + (10)_8 \cdot (3)_8 + (1)_8 \\ &= (1131)_8 \end{aligned}$$

oder

$$(1001011001)_2 = 001|001|011|001 = 1|1|3|1 = (1131)_8$$

d) Zerlegen

$$\begin{aligned} (E2A0)_{16} &= E|2|A|0 = 14_{10}|2_{10}|10_{10}|0_{10} = 1110|0010|1010|0000 \\ &= (1110001010100000)_2 \end{aligned}$$

2. a) Die Addition erfolgt ebenso wir bei vorzeichenlosen Dualzahlen. Ein eventuell auftrettender Übertrag wird ignoriert, sodass das Ergebnis wieder eine Dualzahlen der Länge n ist.

b) Die Subtraktion wird auf Vorzeichenwechsel und Addition zurückgeführt, d.h. $(01110)_2 - (00111)_2 = (01110)_2 + (-(00111))_2$.

$(-(00111))_2 = (11000)_2 + 1 = (11001)_2$

```
   01110
+  11001
--------
  100111
```

Ergibt $(01110)_2 - (00111)_2 = (00111)_2$.

Test (nicht Teil der Aufgabe).
$(01110)_2 = (14)_{10}$, $(00111)_2 = (7)_{10}$, $(14)_{10} - (7)_{10} = (7)_{10}$.

4 Algorithmen (1/2)

1. Behauptung. Die Anzahl der Teile der Länge 1 oder 2, in die ein Faden der positiven ganzzahligen Länge n zerschnitten werden kann, wird durch folgende rekursive Formel bestimmt.

$$T(1) = 1$$
$$T(2) = 2$$
$$T(n) = T(n-1) + T(n-2)$$

   a) Zeigen Sie mit vollständiger Induktion die Korrektheit der Behauptung. | (6 Punkte)

   b) Geben Sie in Pseudo-/Java-Code einen Algorithmus zur iterativen Berechung der Anzahl der Teile der Länge 1 oder 2, in die ein Faden der positiven ganzzahligen Länge n zerschnitten werden kann, an. (6 Punkte)

Lösung.

1. a) Induktionsanfang.
Zu zeigen, die Behauptung gilt für $n = 1$ und $n = 2$.

Ein Faden der Länge 1 kann nur in ein Teil der Länge 1 geteilt werden, $T(1) = 1$.
Ein Faden der Länge 2 kann einmal in zwei Teile der Länge 1 und einmal in ein Teil der Länge 2 geteilt werden, $T(2) = 2$.

Induktionsannahme.
Es gibt ein $n > 2$, sodass die Behauptung für alle Fäden der Länge kleiner gleich n gilt.

Induktionsschritt.
Zu zeigen, mit der Induktionsannahme, gilt die Behauptung auch für Fäden der Länge $n + 1$ mit $n > 2$.

Ein Faden der Länge $n + 1$ kann einmal in ein Teil der Länge 1 und alle Teilungen eines Fadens der Länge n geteilt werden und einmal in ein Teil der Länge 2 und alle Teilungen eines Fadenes der Länge $n - 1$ geteilt werden.
Das heißt $T(n + 1) = 1 \cdot T(n) + 1 \cdot T(n - 1)$, mit Induktionsannahme.

b)

```
1  public long teilungen (int n) {
2      if (n<=0) return 0; // Fehler
3      if (n==1) return 1;
4      if (n==2) return 2;
5      int i = 3;
6      long a = 2;
7      long b = 1;
8      while (i <= n) {
9          long next = a + b;
10         i++;
11         b = a;
12         a = next;
13     }
14     return a;
15 }
```

4 Algorithmen (2/2)

2. Betrachten sie den Java-Programmtext der Funktion `f`.

```
1  public boolean f(int x) {
2      boolean b = true;
3      if ( x <= 1 )          // vorher ( x < 1 )
4          b = false;
5      else if ( x > 1 ) {
6          int t = 2;
7          while ( t*t <= x ) {
8              if ( b && ((x%t) == 0) )
9                  b = false;
10             t++;
11         }
12     }
13     return b;
14 }
```

a) Beschreiben Sie in maximal zwei Sätzen die Funktion der Methode `f`. (4 Punkte)

Hinweis. Welche Eigenschaft der übergebenen Zahl kann aus dem Rückgabewert der Methode gefolgert werden.

b) Definieren Sie für die Schleife Vorbedingung, Nachbedingung und Schleifeninvariante. Beweisen Sie mit deren Hilfe die Korrektheit Ihrer Aussage über die Funktion der Methode `f`. (6 Punkte)

Hinweis. Berücksichtigen Sie auch die Sonderfälle.

Lösung.

2. a) Behauptung. Die Methode `f(x)` liefert *true*, wenn die übergebene Zahl `x` eine Primzahl ist; sonst liefert die Methode *false*.

   b) Vorbedingung. Es gilt `b = true` und keine natürliche Zahl größer gleich 2 und kleiner gleich $t-1$ teilt x ohne Rest.

   Schleifeninvariante. Gilt am Ende jedes Schleifedurchlaufs.
   $(t-1)^2 \leq x$. Genau dann, wenn x von keiner natürliche Zahl größer gleich 2 und kleiner gleich $t-1$ ohne Rest geteilt wird, gilt `b = true` sonst gilt `b = false`.

   Begründung. In der Schleife wird getestet ob x von t ohne Rest geteilt wird, wenn ja wird `b = false` gesetzt, dann wird t um 1 erhöht. D.h. für $t-1$ gilt die Invariante aufgrund dieses Tests und für alle Zahlen kleiner $t-1$ gilt die Invariante wegen der Vorbedingung (beim ersten Durchlauf) oder der Schleifeninvariante (vom vorherigen Durchlauf).

   Nachbedingung. Es gilt `b = true` genau dann, wenn x von keiner natürliche Zahl größer gleich 2 und kleiner gleich $s := (t-1)$ mit $s^2 \leq x$ ohne Rest geteilt wird.

   Beweis der Behauptung. Sonderfall 1. Für 1 (und alle negativen Zahlen) wird *false* zurückgeliefert.

   Sonderfall 2. Für die Primzahlen 2 und 3 wird die Schliefe nicht durchlaufen, aber `b` ist mit *true* initalisiert.

   Für alle $x > 3$ wird die Schleife mindestens einmal durchlaufen.

   Die Nachbedingung stellt sicher, dass x von keiner natürliche Zahl größer gleich 2 und kleiner x ohne Rest geteilt wird, d.h eine Primzahl ist. Denn gäbe es noch einen Teiler r mit $r^2 > x$, dann wäre $x = r \cdot u$ und $u = x/r < x/\sqrt{x} = \sqrt{x}$, somit wäre u ein Teiler von x mit $u^2 \leq x$ und das widerspricht der Nachbedingung.

5 Wachstum von Funktionen

1. Sei $f(n) \in \mathcal{O}(g(n))$ und $a > 0$.

   Beweisen Sie $f(n) \in \mathcal{O}(a \cdot g(n))$. (4 Punkte)

2. Die folgende Tabelle definiert die Primitiven *Zuweisung*, *Sequenz*, *Alternative* und *Schleife* zur Zerlegung von Methoden. Berechnungen und Vergleiche ohne Zuweisung bleiben unberücksichtigt. Weiterhin ordnet die Tabelle jeder Primitive P eine Kostenmaß $T(P)$ zu.

Primitive	*Beispiel für P*	*Kosten T(P)*
Zuweisung	`a = b + c;`	1
Sequenz	P_1; P_2; ⋮ P_m;	$T(P_1) + T(P_2) + \ldots + T(P_m)$

Alternative	`if (<bedingung>)`
	  P_1;
	`else`
	  P_2;
		$\max(T(P_1), T(P_2))$

Schleife	`for(i=0; i<k; i++)` P_1;	$k \cdot T(P_1)$

   Hinweis. Die Rückgabe eines Funktionswertes (`return`) ist eine Zuweisung.

   Betrachten Sie folgende Methode `armstrong` zum Test ob die übergebene Zahl eine Armstrong-Zahl ist.

```
1  public boolean armstrong(int x) {
2      int summe = 0;
3      int zahl = x;
4      if ( zahl > 0 ) {
5          do {
6              int ziffer = zahl % 10;
7              summe = summe + ziffer*ziffer*ziffer;
8              zahl = zahl/10;
9          } while (zahl > 0);
10     }
11     return (x == summe);
12 }
```

   a) Geben sie, basierend auf obigem Kostenmaß, eine Laufzeitfunktion $T_a(x)$ für die Methode `armstrong` an. (6 Punkte)

   b) Geben Sie, mit Herleitung, die asymptotische Laufzeit der Methode `armstrong` in $\mathcal{O}$-Notation an. (6 Punkte)

Lösung.

1. Es gilt

   $f(n) \in \mathcal{O}(g(n)),$ d.h. es gibt $c, n_0 > 0,$ so dass für $n \geq n_0$ gilt $0 \leq f(n) \leq c \cdot g(n)$

   Definiere $d := c/a$, dann folgt für alle $n_0 > 0$

   $$0 \leq f(n) \leq c \cdot g(n) = d\big(a \cdot g(n)\big)$$

   Das heißt $f(n) \in \mathcal{O}(a \cdot g(n))$.

2. a) Die Methode ist eine Sequenz von drei Zuweisungen und einer Alternative.

      Die Alternative hat nur einen `if`–Block, der aus einer Schleife besteht.

      Die Schleife enthät eine Sequenz von drei Zuweisungen und wird sooft durchlaufen, wie x Ziffern hat, d.h. $\lceil \log_{10}(x) \rceil$–mal. Daraus folgt $3\lceil \log_{10}(x) \rceil$ Kosten für die Schleife.

      D.h. $T_a(x) = 3 + 3\lceil \log_{10}(x) \rceil$.

   b) Wähle $x_0 = 10$ und $c = 9$ daraus folgt, für alle $x > x_0$ gilt

      $$0 < 3 + 3\lceil \log_{10}(x) \rceil \quad \text{weil } \lceil \log_{10}(x) \rceil \geq 1 \text{ mit } x \geq 1$$

      und

      $$\begin{aligned} 3 + 3\lceil \log_{10}(x) \rceil &\leq 3 + 3(\log_{10}(x) + 1) \\ &= 6 + 3\log_{10}(x) \\ &\leq 6\log_{10}(x) + 3\log_{10}(x) \quad \text{weil } \log_{10}(x) \geq 1 \text{ mit } x \geq 10 \\ &= 9\log_{10}(x) \\ &\leq c\log_{10}(x) \end{aligned}$$

      Daraus folgt $T_a(x) \in \mathcal{O}(\log_{10}(x))$.

6 Abstrakte Datentypen

Ihnen steht eine Klasse `CharStack`, die einen Kellerspeicher für Zeichen implementiert, mit folgenden Methoden zur Verfügung.

Methode	*Funktion*
`void create`	Konstruktor, erzeugt einen leeren Stapel
`boolean empty`	Liefert *`true`*, wenn der Stapel leer ist, sonst *`false`*.
`void push(char ch)`	Legt das Zeichen `ch` oben auf den Stapel.
`char pop()`	Liefert das oberste Zeichen zurück und entfernt es vom Stapel, ist der Stapel leer wird `null` zurückgeliefert.

Schreiben Sie in Pseudo-/Java-Code eine Methode, die, mit Hilfe der Klasse `CharStack`, einen übergebenen String darauf testet, ob er ein Palindrom ist. (10 Punkte)

Hinweis. Ein Palindrom ist ein Wort bzw. eine Zeichenkette, die vorwärts und rückwärtsgelesen, genau gleich ist. Beispiele. `LAGERREGAL` oder `p#?& &!& &?#p`.

Lösung.

```
1   public boolean palindrom( String s ) {
2
3       Stack stack = new Stack();
4       // oder
5       Stack stack = Stack.create();
6
7       int half = s.length / 2;
8       int plus = s.length % 2;
9
10      for ( int i = 0; i < half; i++ ) {
11          stack.push(s.charAt(i));
12      }
13
14      boolean p = true;
15      for ( int i = half + plus; p && i < args[0].length; i++) {
16          if ( stack.pop() != s.charAt(i) )
17              p = false;
18      }
19
20      return p;
21  }
```

7 Java

Betrachten Sie folgendes Java-Programmfragment.

```
1  for ( i=0; i < n; i++) {
2      if ( a[i] == x)
3          break;
4      else
5          a[i] = 0;
6  }
```

Schreiben Sie das Java-Programmfragment so um, dass anstelle der `for`– eine `do-while`–Schleife und keine `break`–Anweisung benutzt wird. (4 Punkte)

Hinweis. Auch ein leeres Feld ($n = 0$) muß berücksichtigt werden.

Lösung.

1.
```
1  int i = 0;
2  do {
3       if ( (i < n) && (a[i] != x) )
4            a[i] = 0;
5       i++;
6  } while ( (i < n) && (a[i] != x) )
```

Anhang. Exkurs, Äquivalenzumformung

Der ursprüngliche Hinweis zu Aufgabe 2.2 lautete: Beginnen Sie, indem Sie $\wedge\neg x$ auf beide Seiten der Gleichung anwenden.

Anschließend sollte, durch Anwendung der bekannten Gesetze, jede Seite der Gleichung auf 0 zurückgeführt werden, sodass sich die Gleichung $0 = 0$ ergibt.

Diese Verfahren wird *Äquivalenzumformung* einer Gleichungen genannte. Kann eine Gleichung durch Äquivalenzumformungen auf eine korrekt Gleichung (z.B. $0 = 0$) zurückgeführt werden, ist damit die Korrektheit der ursprünglichen Gleichung bewiesen.

Die Anwendung einer *bijektiven Funktion* auf beide Seiten einer Gleichung ist eine Äquivalenzumformung.

Ein Funktion $f : B \to B$ ist bijektiven, wenn es eine Umkehrfunktion $f^{-1} : B \to B$ gibt, für die gilt $f^{-1}(f(b)) = b$ und $f(f^{-1}(b)) = b$

Das Problem ist, dass die Anwendung von $\wedge\neg x$ auf eine beliebige Boolesche Funktion b (z.B. $b(x, y) = x \wedge (x \vee y)$), also $f(b) = b \wedge \neg x$ keine bijektiven Funktion und damit auch keine Äquivalenzumformung ist.

$\mathbb{B} = (\{0, 1\}, \vee, \wedge, \neg)$ ist die Boolesche Algebra. Das neutrale Element der $\vee$–Verknüpfung ist 0 ($x \vee 0 = x$) und das der $\wedge$–Verknüpfung ist 1 ($x \wedge 1 = x$). Aber es gibt weder für die $\vee$– noch für die $\wedge$–Verknüpfung für jedes Element $x \in \{0, 1\}$ eine Inverses-Element, d.h. ein Element, dass verknüpft mit dem ursprünglichen Element das neutrale Element ergibt. Es gibt für 1 keine Inverses unter der $\vee$–Verknüpfung, denn $1 \vee 0 = 1 \vee 1 = 1 \neq 0$. Genauso gibt es keine Inverses für 0 unter der $\wedge$–Verknüpfung, denn $0 \wedge 0 = 0 \wedge 1 = 0 \neq 1$.

Angenommen es gäbe für $f(b) = b \wedge \neg x$ eine Umkehrfunktion f^{-1} mit $f(f^{-1}(b)) = b$, dann würde aus $f(f^{-1}(1)) = 1$ folgen $f^{-1}(1) \wedge \neg x = 1$, d.h. bedeutet $f^{-1}(1)$ ist das Inverse zu $\neg x$ unter der $\wedge$–Verknüpfung und das ist eine Widerspruch.

Bijektive Funktion sind z.B. $g(b) = g^{-1}(b) = \neg b$ oder $h(b) = h^{-1}(b) = (\neg x \wedge b) \vee (x \wedge \neg b)$.

Mit h läßt sich das Absorptionsgesetz zeigen.

	$x \vee (x \wedge y) = x$	
$\Leftrightarrow$	$h\big(x \vee (x \wedge y)\big) = h\big(x\big)$	Anwendung von h
$\Leftrightarrow$	$h\big(x \vee (x \wedge y)\big) = (\neg x \wedge x) \vee (x \wedge \neg x)$	
$\Leftrightarrow$	$h\big(x \vee (x \wedge y)\big) = 0$	Komplementär-
$\Leftrightarrow$	$(\neg x \wedge (x \vee (x \wedge y))) \vee (x \wedge \neg(x \vee (x \wedge y))) = 0$	
$\Leftrightarrow$	$(\neg x \wedge (x \vee (x \wedge y))) \vee (x \wedge (\neg x \wedge \neg(x \wedge y))) = 0$	De-Morgan
$\Leftrightarrow$	$(\neg x \wedge (x \vee (x \wedge y))) \vee ((x \wedge \neg x) \wedge \neg(x \wedge y))) = 0$	Assoziativ-
$\Leftrightarrow$	$(\neg x \wedge (x \vee (x \wedge y))) \vee (0 \wedge \neg(x \wedge y))) = 0$	Komplementär-
$\Leftrightarrow$	$\neg x \wedge (x \vee (x \wedge y)) = 0$	Extremal-, Neutralitäts-
$\Leftrightarrow$	$(\neg x \wedge x) \vee (\neg x \wedge (x \wedge y)) = 0$	Distributiv-
$\Leftrightarrow$	$0 \vee ((\neg x \wedge x) \wedge y)) = 0$	Komplementär-, Assoziativ-
$\Leftrightarrow$	$0 = 0$	Komplementär-, Extremal-

Das ist aber unnötig kompliziert, die Fallunterscheidung ist deutlich effizienter.